UNIVERSITY OF Southampton
School of Electronics and Computer Science

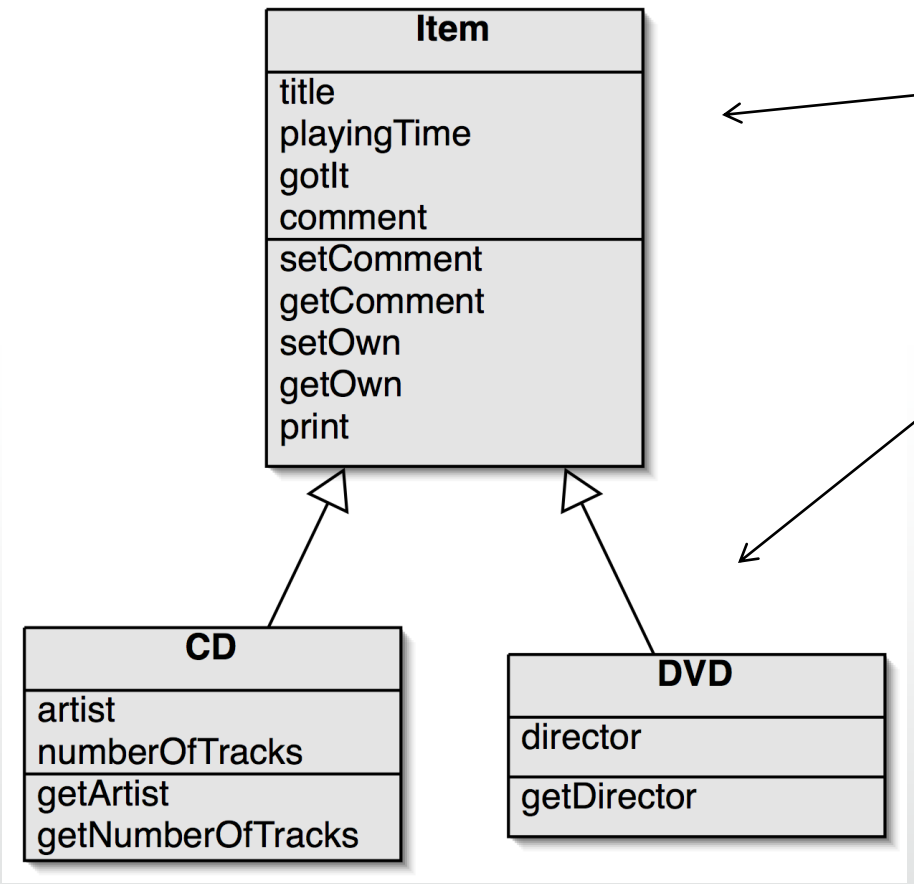
Comp1004: Inheritance II

Polymorphism

Coming up

- Inheritance Reminder
- Overriding methods
  - Overriding and substitution
- Dynamic Binding
- Polymorphism
  - Designing for Polymorphism
  - The super keyword

Inheritance Reminder

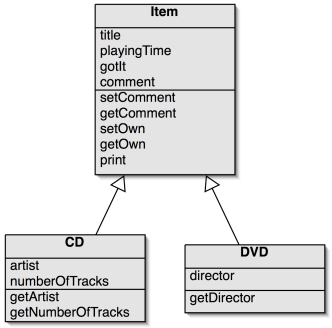


Any class that is inherited from is called a **superclass**

Any class that inherits from another is called a **subclass**

The sub-classes inherit all the properties and methods from the superclass

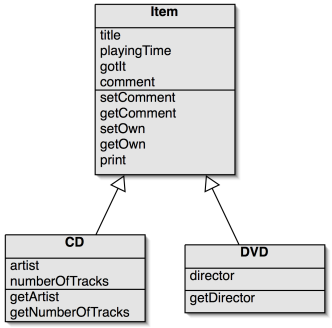
They can also add more of their own



Inheritance in Java

```
public class Item
{
    private String title;
    private int playingTime;
    private boolean gotIt;
    private String comment;

    // constructors and methods omitted.
}
```

You don’t need to add anything to the superclass



Inheritance in Java

```
public class CD extends Item
{
    private String artist;
    private int numberOfTracks;

    // constructors and methods omitted.
}
```

You don't need to add anything to the superclass

You declare the inheritance in the sub-class using the **extends** keyword

```
public class DVD extends Item
{
    private String director;

    // constructors and methods omitted.
}
```

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation
- Public
  - Everyone can see it
- Protected
  - Only *this class, its sub-classes and the package* can see it
- Default (no keyword)
  - Only *this class and the package* can see it
- Private
  - Only *this class* can see it

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation
- Public
  - Everyone can see it
- Protected
  - Only *this class, its sub-classes and the package* can see it
- Default (no keyword)
  - Only *this class and the package* can see it
- Private
  - Only *this class* can see it

Rule-of-thumb: assume everything should be protected

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation

- Public
  - Everyone can see it

- Protected
  - Only *this class, its sub-classes and the package* can see it

- Default (no keyword)
  - Only *this class and the package* can see it

- Private
  - Only *this class* can see it

Explicitly make public the methods (and sometimes properties) that you want other classes to use

Rule-of-thumb: assume everything should be protected

Explicitly make private any implementation details that you want to hide even from sub-classes

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation

- Public
  - Everyone can see it

- Protected
  - Only *this class, its sub-classes and the package* can see it

- Default (no keyword)
  - Only *this class and the package* can see it

- Private
  - Only *this class* can see it

Explicitly make public the methods (and sometimes properties) that you want other classes to use

You will probably end up with few protected things at all!

Explicitly make private any implementation details that you want to hide even from sub-classes

Overriding Methods

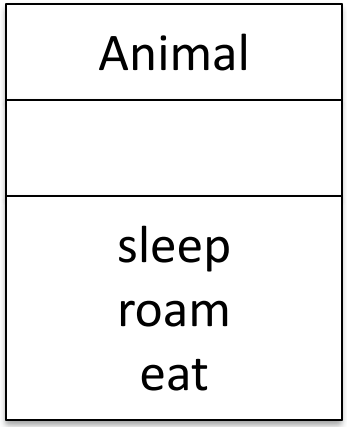
The Animal Class

```
public class Animal
{
        // constructors omitted

        public void sleep() {
                System.out.println("zzz");
        }

        public void roam() {
                System.out.println("wanders about");
        }

        public void eat() {
                System.out.println("eats
something");
        }

}
```

A simple Animal class.

Here the three methods just print out what happens when an animal **sleeps**, **roams** or **eats**

An Animal Class Hierarchy



Dog

- Is a type of Animal
- Dogs sleep like any other animal
- But
  - eats meat and
  - roams in packs
- They have their own way of doing those things
- So we give them their own methods which are more specific than the animal class

The Dog Class

```
public class Dog extends Animal
{
        // constructors omitted

        public void roam() {
                System.out.println("roams in
packs");
        }

        public void eat() {
                System.out.println("eats meat");
        }

}
```

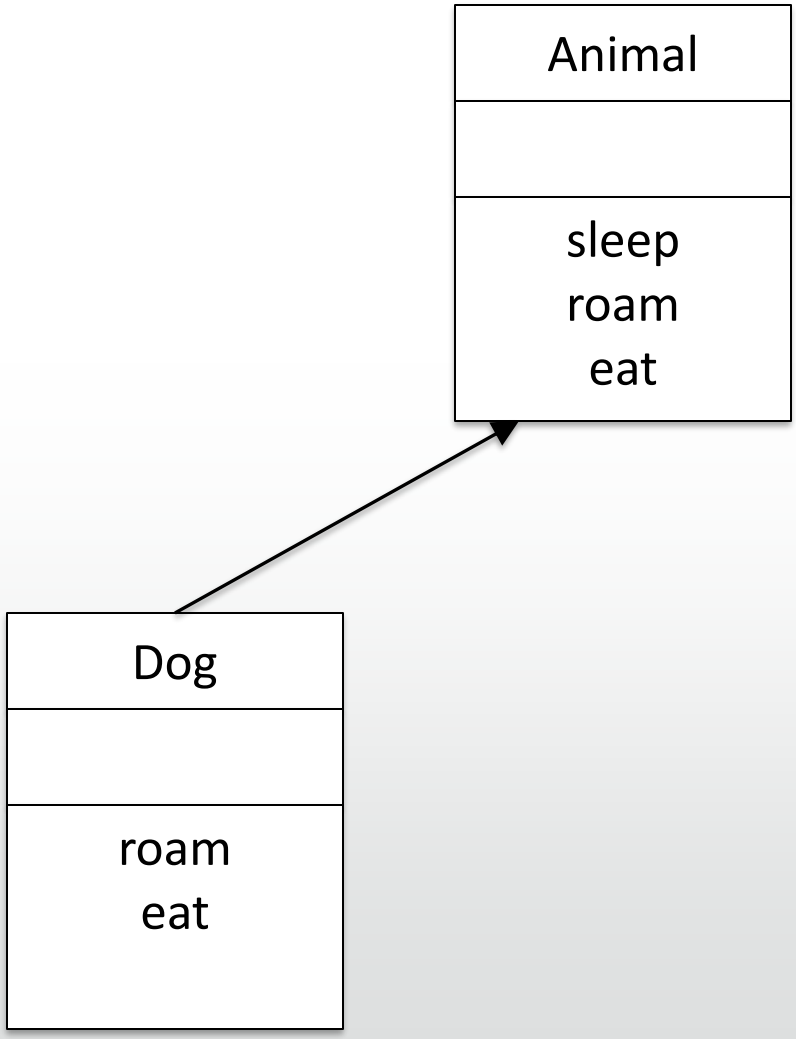
The Dog inherits the sleep method from Animal

But defines its own versions of roam and eat

We say that it **overrides** the roam and eat method from Animal

This means it replaces them with more specific methods for itself

An Animal Class Hierarchy



The Cat Class

```
public class Cat extends Animal
{
        // constructors omitted

        public void roam() {
                System.out.println("roams
independently");
        }

        public void eat() {
                System.out.println("eats meat");
        }

}
```

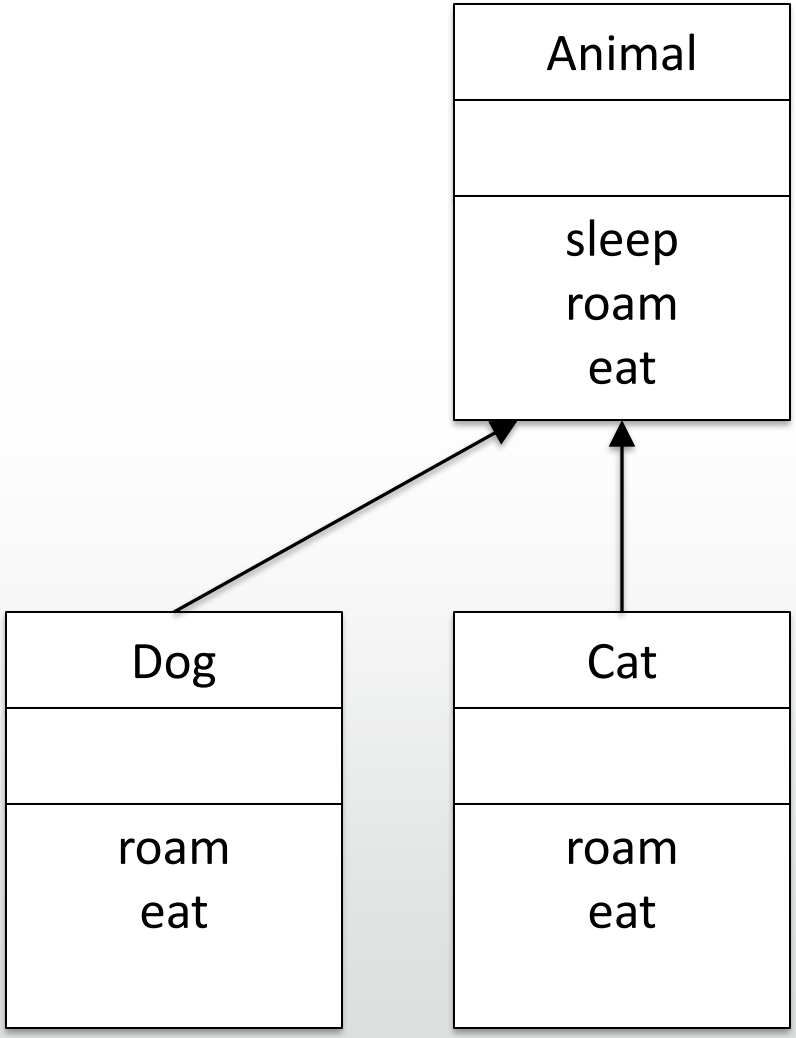
The Cat inherits the sleep method from Animal

But defines its own versions of roam and eat

We say that it **overrides** the roam and eat method from Animal

This means it replaces them with more specific methods for itself

An Animal Class Hierarchy



The Elephant Class

```
public class Elephant extends Animal
{
        // constructors omitted

        public void eat() {
                System.out.println("eats grass");
        }

}
```

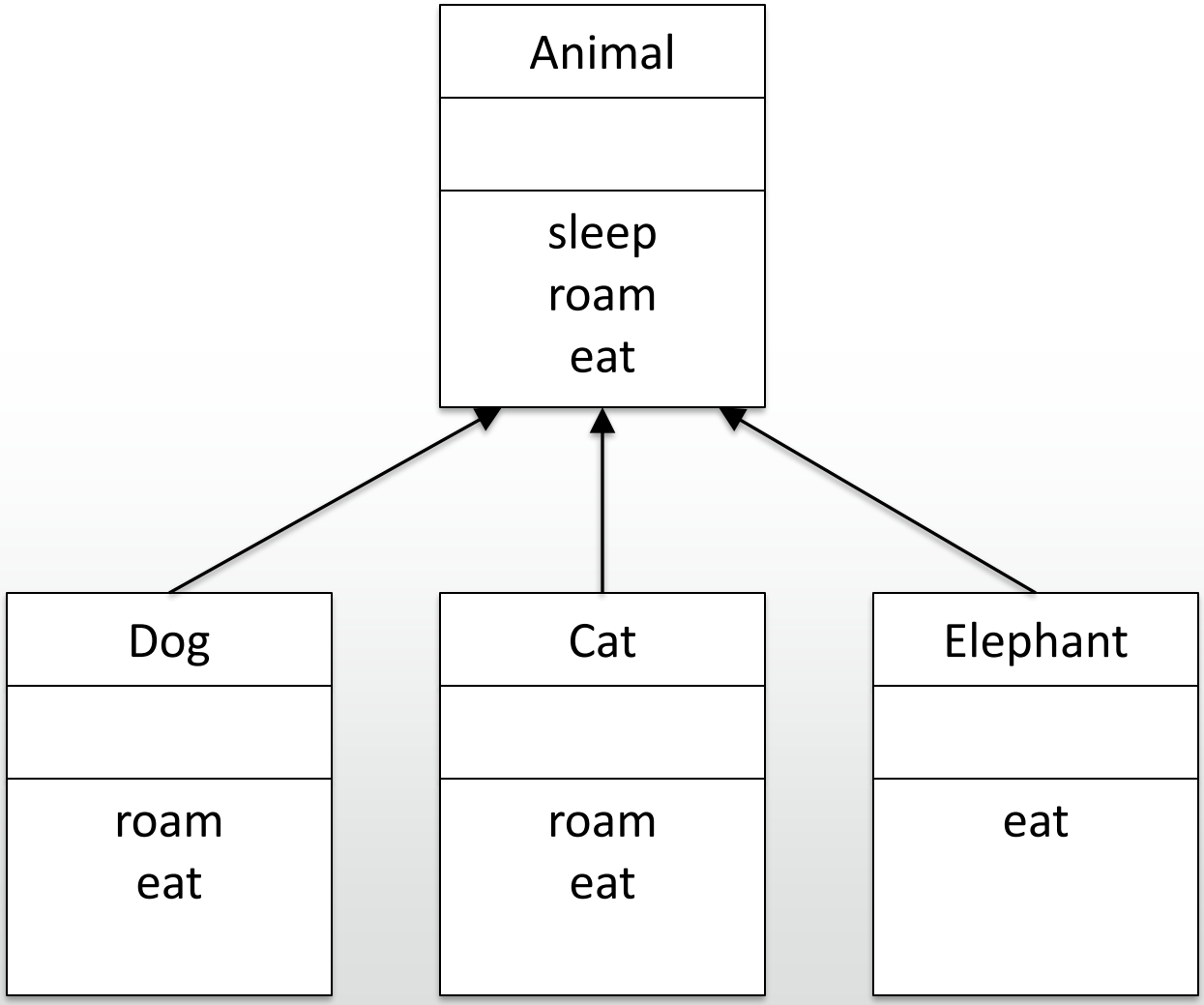
The Elephant inherits the sleep and roam methods from Animal

But defines its own versions of eat

We say that it **overrides** the eat method from Animal

This means it replaces it with a more specific method for itself

An Animal Class Hierarchy



Overriding

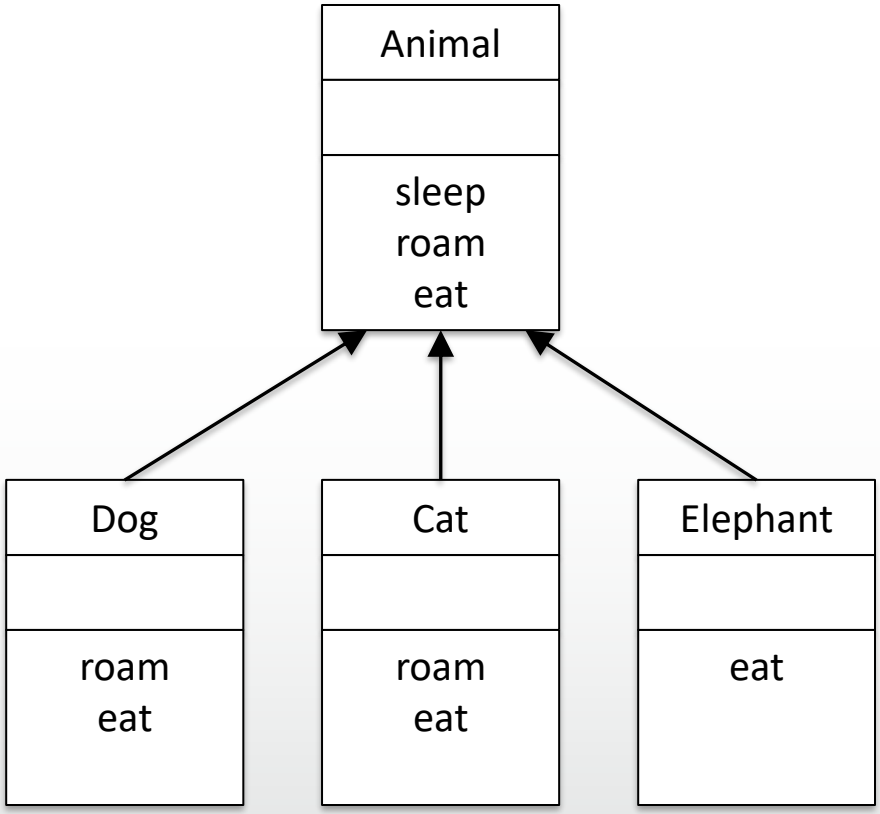
- Replacing a super-class' method with one in its sub-class is called **overriding**

- This is **not** the same as **overloading**
  - Overloading is writing two or more methods in the same class with the same name but different **signatures** (return types and parameter lists)

- When **overriding** the signatures must be the same

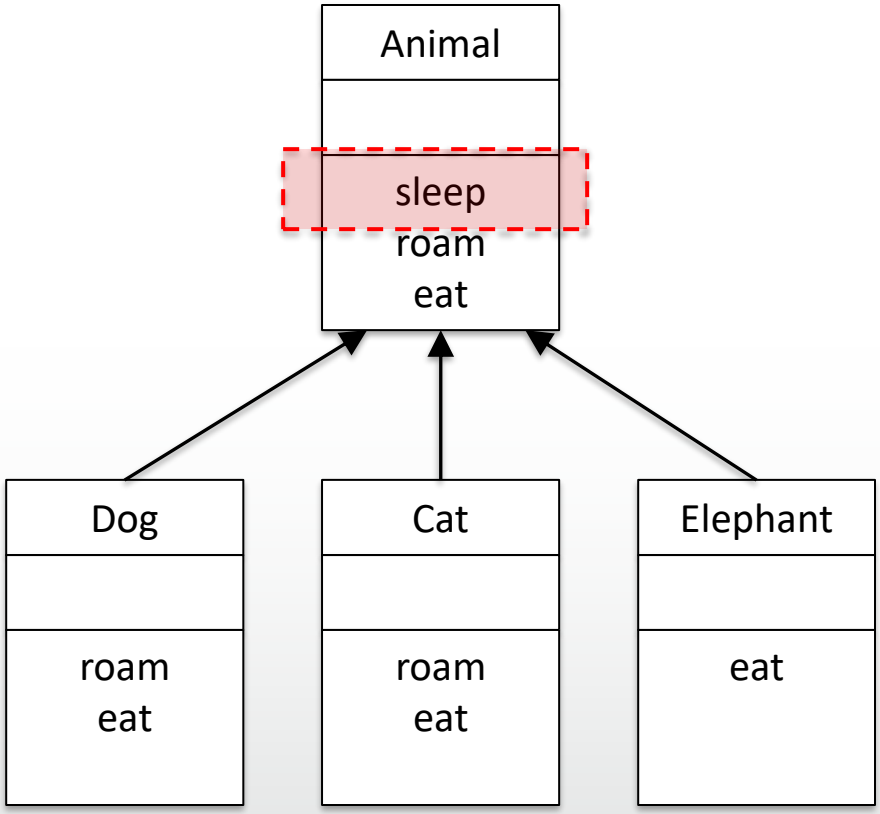
Which method gets called?

```
Animal a = new Animal();
a.sleep();
```



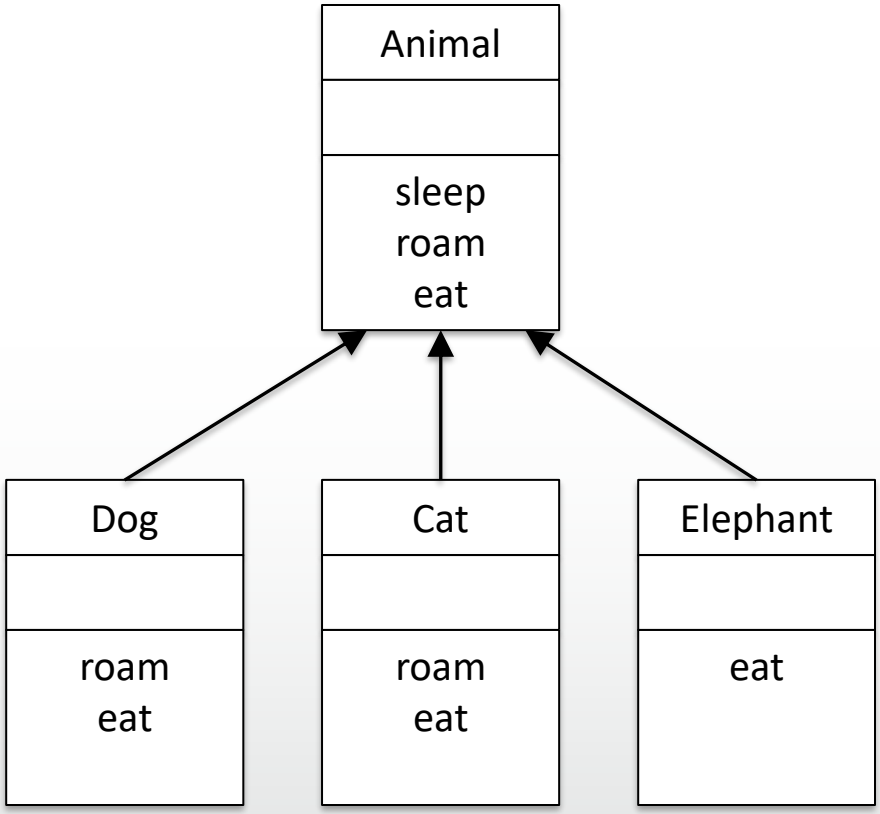
Which method gets called?

```
Animal a = new Animal();
a.sleep();
```



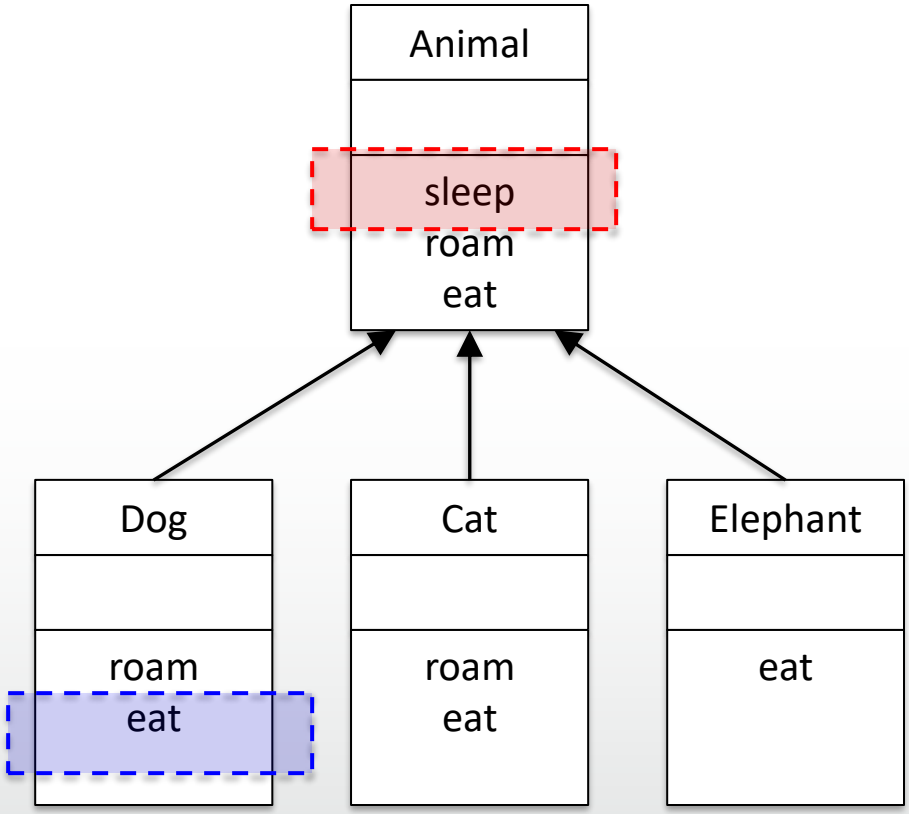
Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();
```



Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();
```



Overriding

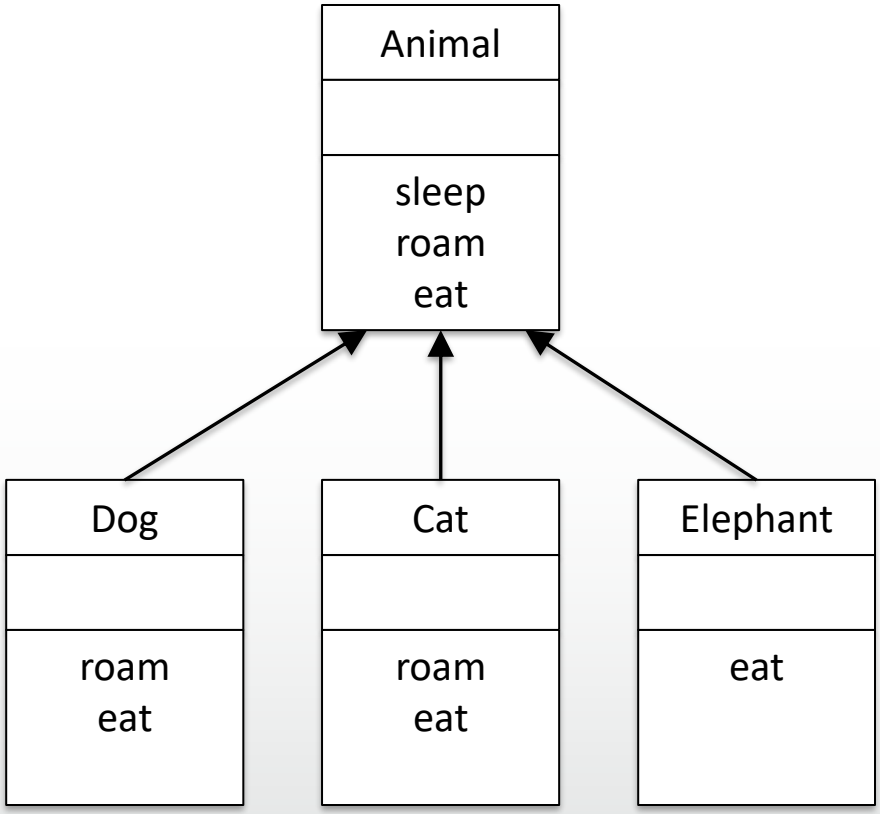
- The first method found from the bottom of the inheritance tree is called

- This means you can override methods from almost all the classes
  - Although the keyword **`final`** will stop you overriding methods. We will study it later.

Dynamic Binding

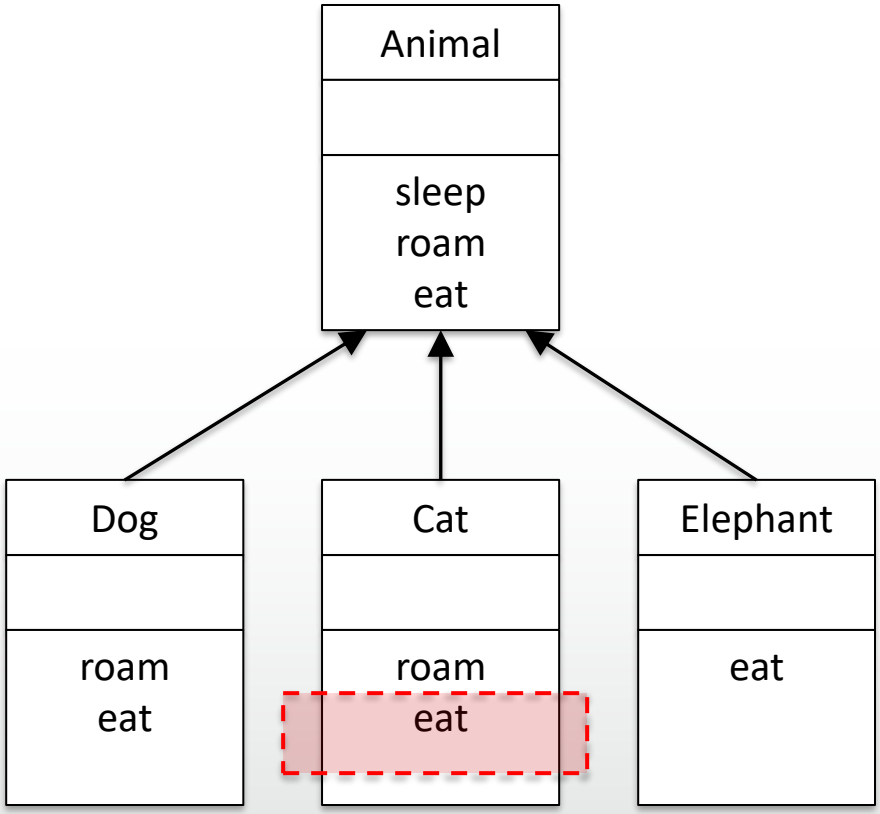
Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();

Animal c = new Cat();
c.eat();
```



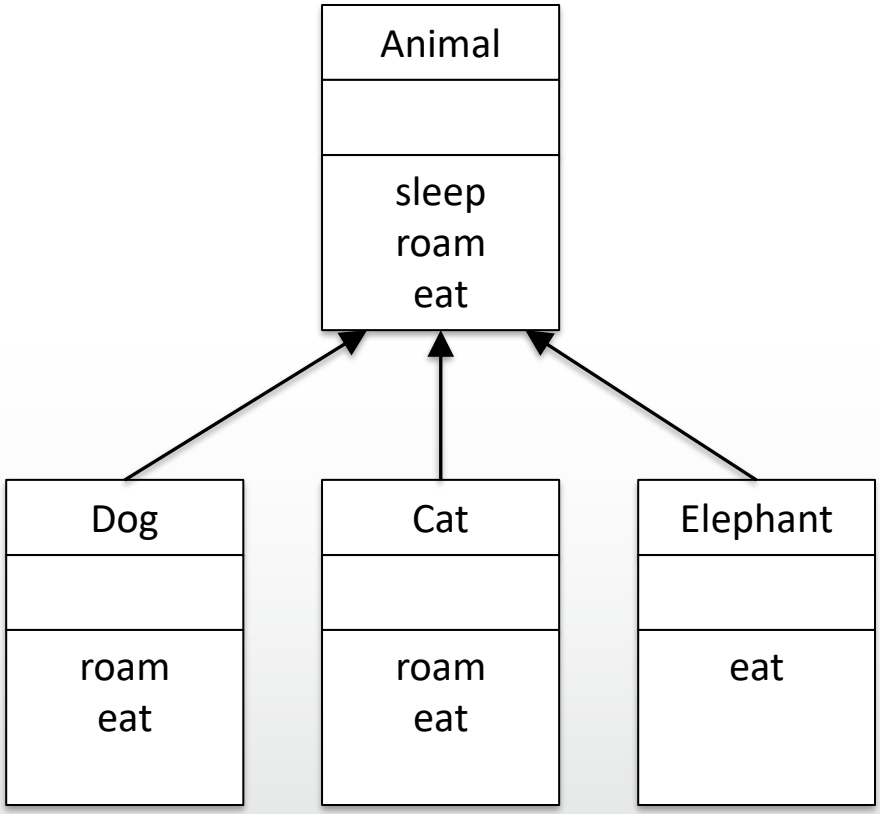
Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();

Animal c = new Cat();
c.eat();
```



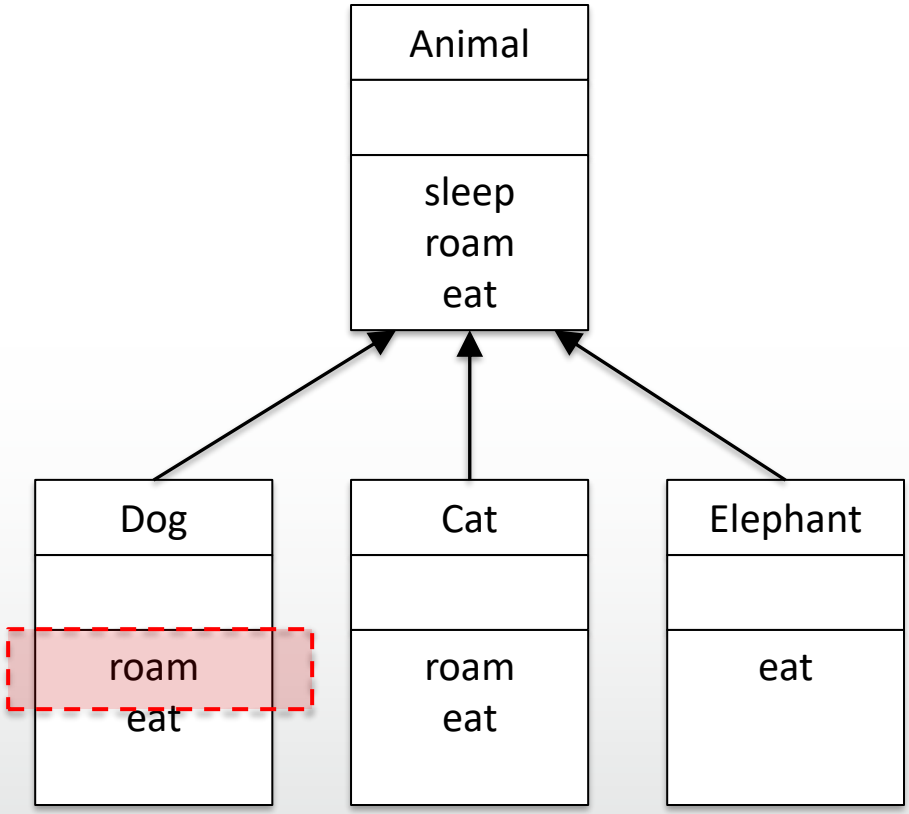
Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();

Animal c = new Cat();
c.eat();

Animal d2 = d;
d2.roam();
```



Which method gets called?

```
Animal a = new Animal();
a.sleep();

Dog d = new Dog();
d.sleep();
d.eat();

Animal c = new Cat();
c.eat();

Animal d2 = d;
d2.roam();
```



Dynamic Binding

- Java is calling the most specific version of the method, even when the reference has the type of the superclass

- This is called dynamic binding, because Java doesn’t figure out which version of the method to call until runtime

Dynamic Binding is **Very** Useful

```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());
```

Remember that we can use a subclass whenever a reference or collection is expecting the superclass – this is called **substitution**

Dynamic Binding is **Very** Useful

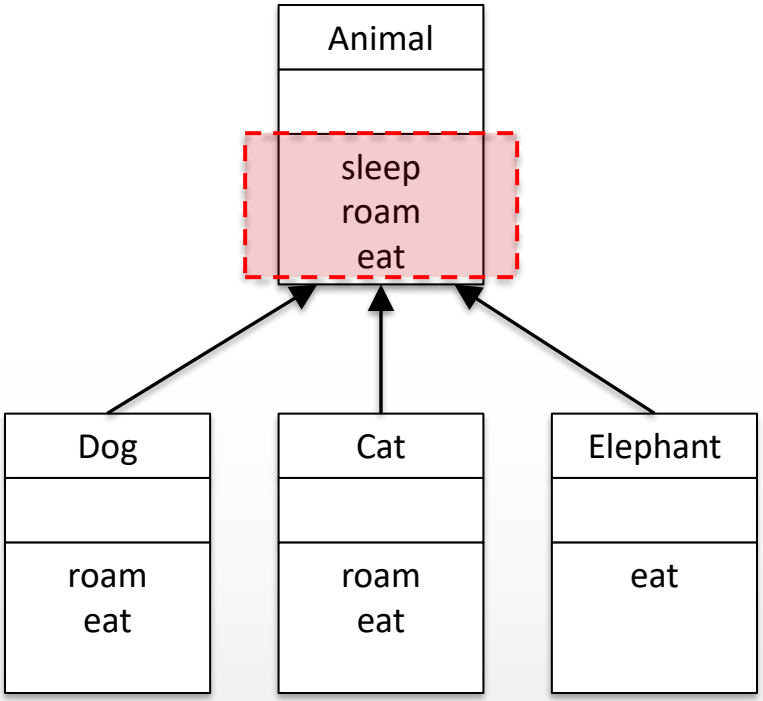
```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());

for(Animal a : animals) {
      a.sleep();
      a.eat();
      a.roam();
}
```

Remember that we can use a subclass whenever a reference or collection is expecting the superclass – this is called **substitution**

Because of **dynamic binding** these will call the most specific version of the method, even though we iterating through an Animal collection

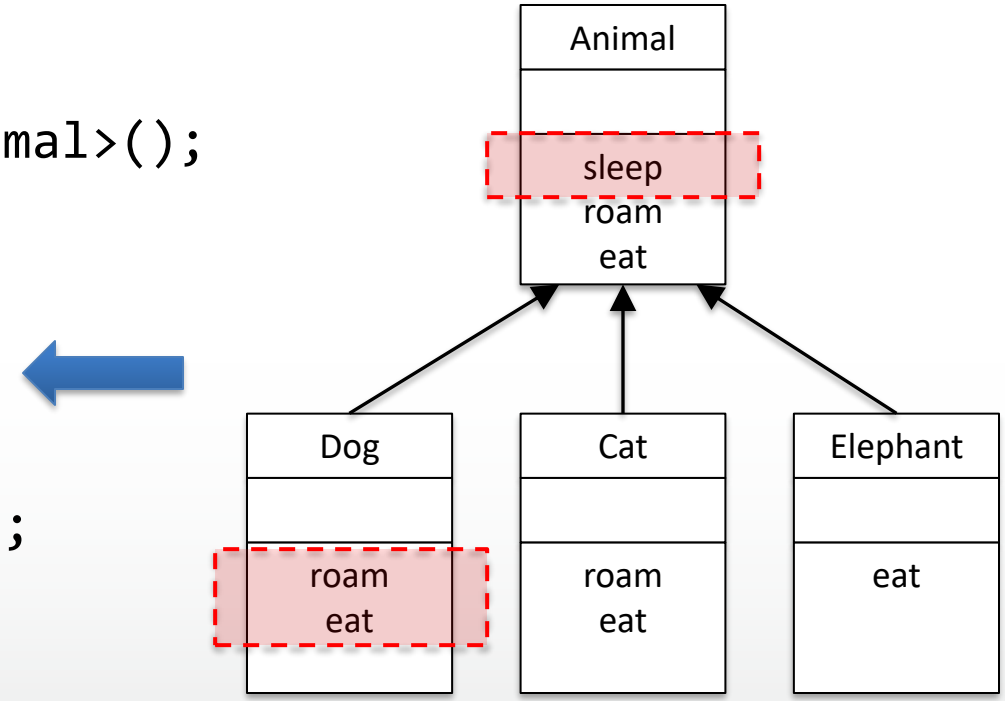
Dynamic Binding is **Very** Useful

```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());

for(Animal a : animals) {
        a.sleep();
        a.eat();
        a.roam();
}
```



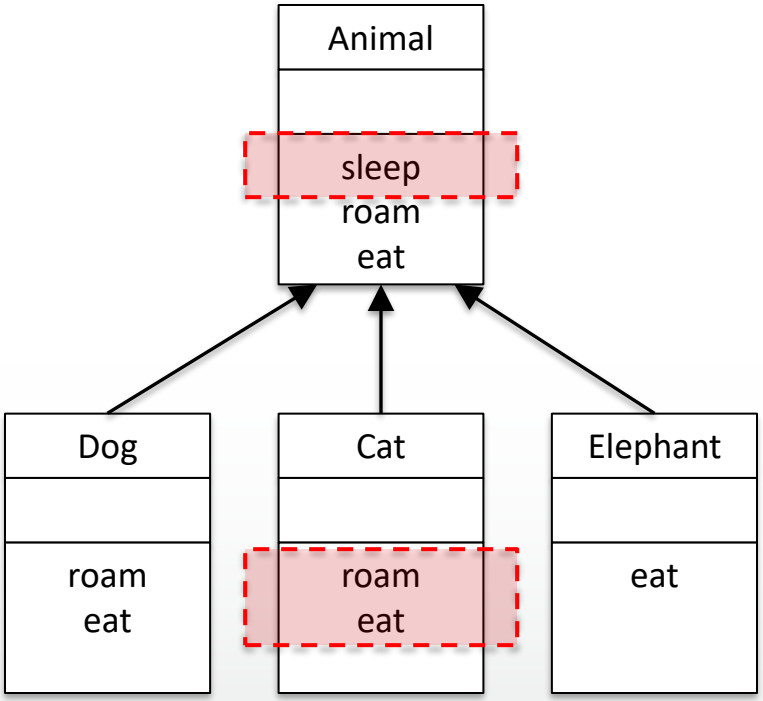
Dynamic Binding is **Very** Useful

```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());

for(Animal a : animals) {
        a.sleep();
        a.eat();
        a.roam();
}
```



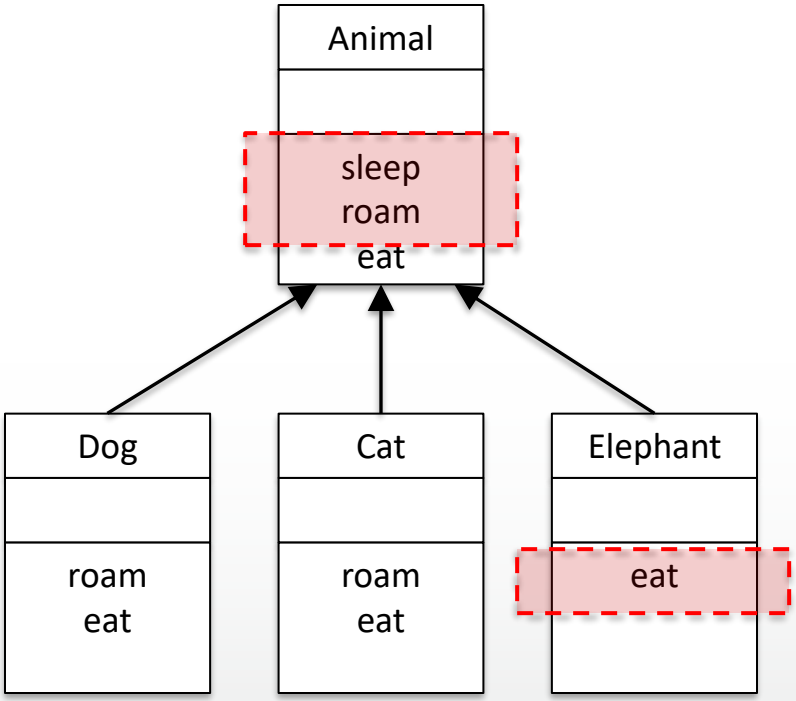
Dynamic Binding is **Very** Useful

```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());

for(Animal a : animals) {
        a.sleep();
        a.eat();
        a.roam();
}
```



Dynamic Binding is **Very** Useful

```
ArrayList<Animal> animals;
animals = new ArrayList<Animal>();

animals.add(new Animal());
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Elephant());

for(Animal a : animals) {
        a.sleep();
        a.eat();
        a.roam();
}
```



Polymorphism

Polymorphism

- Substitution, Overriding and Dynamic Binding gives us **Polymorphism**
  - *(greek for many shapes)*

- Polymorphism means that we can create methods that deal with superclasses and then:
  - We can pass them instances of sub-classes (*substitution*)
  - And when it calls methods on those sub-classes if there are more specific methods defined (via *overriding*) the call gets diverted at run-time to the most specific method (*dynamic binding*)

Example: Object.toString()

- Methods in `Object` are inherited by all classes.
  - Any of these may be overridden.

- The `toString` method is commonly overridden:
  - `public String toString()`
  - Returns a string representation of the object.

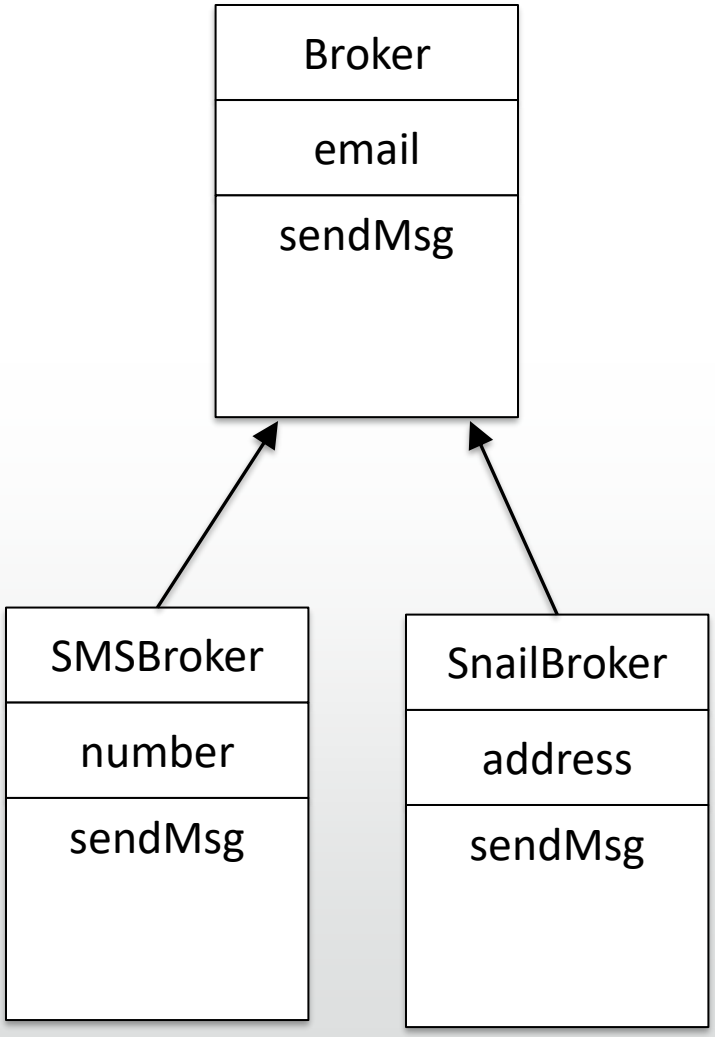
- Calls to `println` automatically result in `toString` being called in the appropriate sub-class:
  - `System.out.println(item);`

Designing for Polymorphism

“We have a system that needs to send messages to people. Usually we contact people via email, but some like to be contacted via SMS and some via snail mail. What sort of class structure might we build?”

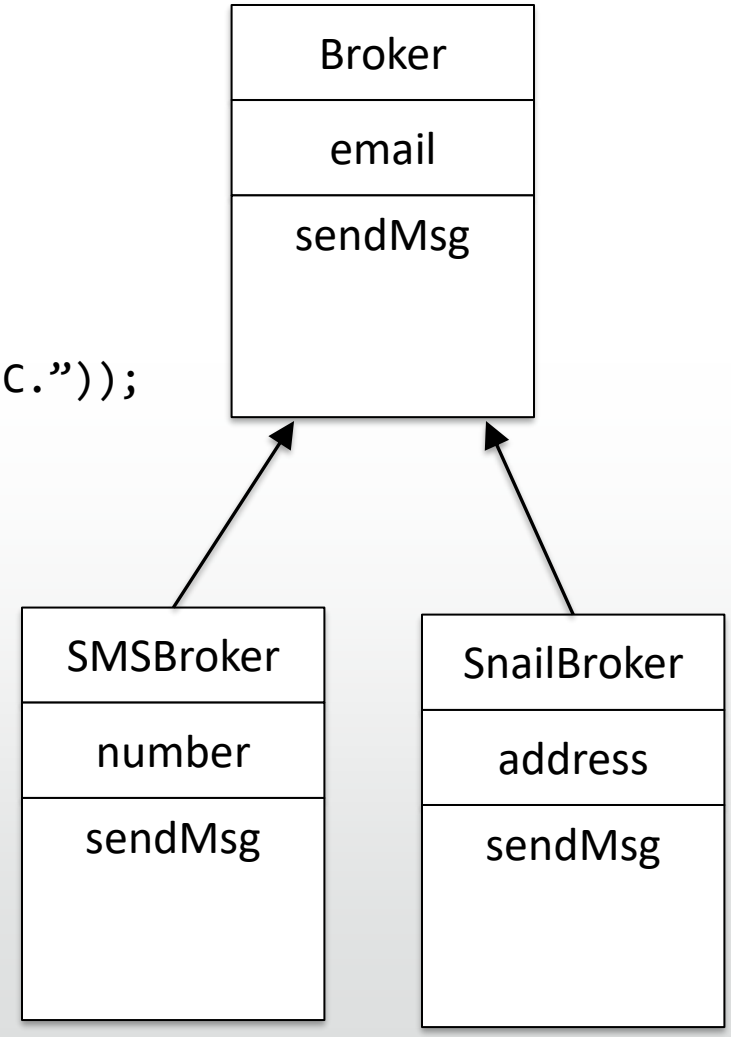
Designing for Polymorphism

"We have a system that needs to send messages to people. Usually we contact people via email, but some like to be contacted via SMS and some via snail mail. What sort of class structure might we build?"



Designing for Polymorphism

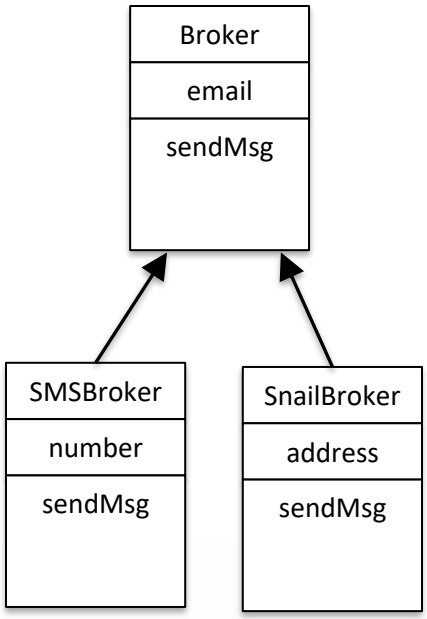
```
ArrayList<Broker> brokers;
Brokers = new ArrayList<Broker>();

brokers.add(new Broker("pm@numberten.gov.uk"));
brokers.add(new Broker("pres@france.gov.uk"));
brokers.add(new SMSBroker("07123 456789"));
brokers.add(new SnailBroker("1600 Penns Ave. D.C."));
brokers.add(new SMSBroker("07987 654321"));

String message = "Reminder. Fix economy!");

for(Broker b : brokers) {
        b.sendMsg(message);
}
```



The Three Pillars of OOP

Encapsulation Inheritance Polymorphism

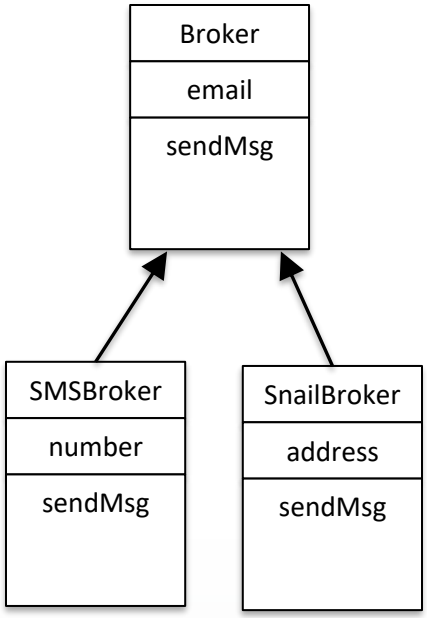
The super keyword




```
public class Broker{
   //code omitted

   public void sendMsg(String msg) {
        //code to send email omitted
   }
}

public class SMSBroker extends Broker{
   //code omitted

   public void sendMsg(String msg) {
        //code to send SMS omitted
   }
}
```

What happens if we want to access a method in the superclass – i.e. perhaps as well as sending an SMS the SMSBroker also wants an email?

The super keyword




```
public class Broker{
   //code omitted

   public void sendMsg(String msg) {
        //code to send email omitted
   }
}

public class SMSBroker extends Broker{
   //code omitted

   public void sendMsg(String msg) {
        super.sendMsg(msg);
        //code to send SMS omitted
   }
}
```

What happens if we want to access a method in the superclass – i.e. perhaps as well as sending an SMS the SMSBroker also wants an email?

Within a sub-class we can use **super** to access protected or public methods in the superclass

Summary

- Inheritance Reminder
- Overriding methods
  - Overriding and substitution
- Dynamic Binding
- Polymorphism
  - Designing for Polymorphism
  - The super keyword